

0.1 Chapter 1 — Introduction

This thesis studies how to **accelerate test-time discovery in complex code generation via execution-guided self-distillation**. The core question is practical: when a language model is given a single hard programming problem and a budget to improve itself at inference time, what is the most effective way to turn execution feedback into a better solution, and when does that strategy work?

0.1.1 1.1 Background

Reinforcement learning with verifiable rewards (RLVR) is the standard recipe for post-training language models on code and math, with GRPO [8] as a common instantiation. Its weakness is sparse credit assignment: a scalar outcome reward says only whether an attempt passed, not where it went wrong, and when every sampled rollout fails, the advantage collapses and learning stalls. This failure is most acute on exactly the problems that matter, the hard ones, where successes are rare.

Self-Distillation Policy Optimization (SDPO) [1] addresses this by extracting a denser signal from the *rich feedback* that verifiable environments already produce but usually discard, such as runtime errors and failing tests. The same model, conditioned on this feedback, acts as a self-teacher whose retrospective per-token distribution is distilled back into the student. At test time, SDPO can iterate on a single hard problem and improve before it ever produces a first success, measured by `discovery@k`, the probability of solving within the first k attempts [1, §5].

Two open issues motivate this work. First, the parent paper organizes its update *student-first*: it distills the student’s own failed rollout. On a problem the bare student rarely solves, those rollouts are almost never correct, so there is little that is right to learn from. Second, Kim et al. [2] show that distilling from rich context can degrade a model by suppressing its epistemic verbalization, which raises the question of whether the feedback helps the model reason or merely teaches it to copy.

0.1.2 1.2 Problem

The setting is test-time training (TTT): one hard problem, a verifiable reward, and a short budget of self-improvement steps, after which the model is reset. Two design choices govern how well the model discovers a solution. One is *how the execution feedback is organized into a learning signal*: should the model learn from its own failed attempt, or from a correct attempt produced under feedback and filtered for independence? The other is *how the execution feedback is formulated* into the reprompt the self-teacher sees, which the parent paper leaves as an explicit open question [1, §7]. This thesis takes execution feedback as the privileged context that guides self-distillation, hence *execution-guided*, and studies both choices with the goal of accelerating discovery.

0.1.3 1.3 Research questions

Two primary questions drive the study:

- **RQ-method.** At test time, does a teacher-first organization, which distills filtered teacher-generated trajectories, beat the student-first baseline, which distills the student’s own failed rollout, at discovering solutions to hard code problems?

- **RQ1.** Does the reprompt-template formulation of the execution feedback affect discovery, and in which regime?

Two secondary, exploratory questions situate the work within the wider self-distillation literature:

- **RQ2 (exploratory).** Does self-distillation from rich context suppress epistemic verbalization at test time? Only a limited math pilot bears on this, since the code runs produce no reasoning trace.
- **RQ3 (exploratory).** How does discovery trade off against compute? This thesis quantifies the compute overhead of teacher-first and identifies the compute-matched comparison a full answer requires; the compute-to-correct Pareto itself is left to future work (§4.2.5, §6.2.4).

0.1.4 1.4 Contributions

The contributions are stated at the strength the evidence supports; the sample sizes are small and the claims are directional, never proofs.

1. **A teacher-first variant of execution-guided self-distillation.** A method that distills the student toward verifier- and judge-filtered trajectories generated by the self-teacher, positioned between on-policy SDPO and off-policy SFT (Chapter 3).
2. **Evidence that teacher-first accelerates discovery where on-policy SDPO stalls.** Across matched comparisons, teacher-first is at least as good as student-first on every seed and never worse, and on the hardest problems it escapes the flat-reward trap that keeps student-first stuck at zero. This escape-zero behavior is replicated three times, with one case improving the pass-rate roughly ninefold (Chapter 4).
3. **A reprompt-template effect.** The formulation of the execution feedback affects discovery, most clearly on hard problems, where a reasoning-inducing template orders above the plain anchor, which orders above a minimal one (§4.3).
4. **A value-versus-procedure boundary characterization.** A math pilot shows where the method does *not* help: when the reference encodes only an answer rather than an in-reach method, the teacher copies the answer (a copy rate near 75% is measured), the leak transfers reasoning style but not capability, and the student stays at zero. This characterizes escape as requiring reachable capability and a method-bearing reference, not merely a correct answer (§4.5, Chapter 5).

0.1.5 1.5 Scope

The study is confined to the test-time training regime: single-problem self-improvement with a per-problem reset, using LoRA adapters, which is the appropriate lightweight choice for adapting to one problem at inference time. Code generation on LiveCodeBench v6 [6] is the core, carrying the main claims; an AIME 2026 math pilot serves as a contrast that characterizes the boundary of the mechanism. Results are on one 4B-scale model per domain, an anchor template for the main comparison, and a 15-step budget. No claim of universality is made.

0.1.6 1.6 Thesis structure

Chapter 2 reviews the background and related work: RLVR and GRPO, SDPO and its loss, test-time training and discovery@k, the reprompt-template design space, and the epistemic-suppression literature. Chapter 3 specifies the method, including the student-first baseline, the proposed teacher-first organization, the template taxonomy, the two domains, and the metrics. Chapter 4 reports the experiments: the main teacher-first versus student-first comparison and the escape-zero result, the template study, the robustness ablations, and the math pilot. Chapter 5 discusses the mechanism and the value-versus-procedure boundary. Chapter 6 states the limitations and the future experiments that would address them. Chapter 7 concludes.

0.1.7 In-chapter citations

Numbered per the master bibliography (`00_references.md`): [1] Hübötter et al. 2026 ð [2] Kim et al. 2026 ð [6] LiveCodeBench ð [8] GRPO / DeepSeekMath.